

✓ Toxic-Comment Paper — Results Notebook (Google Colab)

What this does: trains every model in the paper and prints a clean report + saves `results.json`. Send that file (or the printed **FINAL REPORT** block) back and every red placeholder in the manuscript gets filled with these real numbers.

How to run

1. **Runtime** → **Change runtime type** → **Hardware accelerator** → **GPU** → **Save**.
2. Run each cell top to bottom (Shift+Enter).
3. In the **Data** cell, either upload `train.csv` or use the Kaggle API.
4. When it finishes, download `results.json` (last cell) and share it.

Nothing here is fabricated — the notebook measures everything on the real dataset. Numbers are produced, not invented.

Tip: a full run (all models × several seeds) can take 1–3 h on a Colab GPU. The notebook **saves after every run and resumes** if the session drops, so you can just re-run the experiment cell to continue where it left off.

✓ 1 · Setup & GPU check

```
# Install the one extra dependency (rest is pre-installed on Colab)
!pip -q install iterative-stratification==0.1.7
import tensorflow as tf
print("TensorFlow:", tf.__version__)
gpus = tf.config.list_physical_devices('GPU')
print("GPUs visible to TF:", gpus)
if not gpus:
    print("\n*** NO GPU! Go to Runtime > Change runtime type > GPU, then re-run. ***")
```

```
TensorFlow: 2.20.0
GPUs visible to TF: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

✓ 2 · Capture environment (auto-fills the reproducibility placeholders)

```
import subprocess, sys, json, platform
ENV = {}
ENV["python"] = sys.version.split()[0]
ENV["tensorflow"] = tf.__version__
try:
    import sklearn, numpy, pandas, scipy
    ENV["scikit_learn"] = sklearn.__version__
    ENV["numpy"] = numpy.__version__; ENV["pandas"] = pandas.__version__
    ENV["scipy"] = scipy.__version__
except Exception as e:
    print(e)
try:
    smi = subprocess.check_output(["nvidia-smi", "--query-gpu=name,memory.total,driver_version",
                                   "--format=csv,noheader"]).decode().strip()
    ENV["gpu"] = smi
except Exception:
    ENV["gpu"] = "unknown (no nvidia-smi)"
try:
    ENV["cuda"] = tf.sysconfig.get_build_info().get("cuda_version", "unknown")
    ENV["cudnn"] = tf.sysconfig.get_build_info().get("cudnn_version", "unknown")
except Exception:
    pass
print(json.dumps(ENV, indent=2))
```

```
{
  "python": "3.13.15",
  "tensorflow": "2.20.0",
  "scikit_learn": "1.6.1",
  "numpy": "2.1.3",
  "pandas": "2.2.3",
  "scipy": "1.16.3",
```



```

STATS={"N":int(N),"labels":{}}
print(f"{'label':14s}{ 'n':>8s}{ 'pos%':>8s}{ 'rho':>9s}")
for i,lab in enumerate(LABELS):
    n=int(Y[:,i].sum()); STATS["labels"][lab]={"n":n,"pos_pct":100*n/N,"rho":(N-n)/n}
    print(f"{'lab':14s}{n:8d}{100*n/N:8.2f}{(N-n)/n:9.1f}")
card=float(Y.sum(1).mean()); zero=float((Y.sum(1)==0).mean())
STATS.update(cardinality=card, density=card/L, zero_label_frac=zero,
             avg_pos_rate=float(Y.mean()), naive_accuracy=float(1-Y.mean()))
# token-length percentiles -> sanity-check tmax
lens=df["clean"].str.split().map(len)
STATS["len_median"]=float(lens.median()); STATS["len_p95"]=float(lens.quantile(.95))
STATS["len_p99"]=float(lens.quantile(.99))
print(f"ncardinality={card:.4f} density={card/L:.4f} zero-label frac={zero:.4f}")
print(f"avg pos rate={100*Y.mean():.2f}% naive all-neg accuracy={100*(1-Y.mean()):.2f}%")
print(f"token length: median={lens.median():.0f} p95={lens.quantile(.95):.0f} p99={lens.quantile(.99):.0f}")
print(f"chosen tmax={CONFIG['tmax']} covers {100*(lens<=CONFIG['tmax']).mean():.1f}% without truncation")

# co-occurrence matrix (for the heatmap)
co=np.zeros((L,L),int)
for i in range(L):
    for j in range(L):
        co[i,j]=int(((Y[:,i]==1)&(Y[:,j]==1)).sum())
STATS["cooccurrence"]={LABELS[i]:{LABELS[j]:int(co[i,j]) for j in range(L)} for i in range(L)}
STATS["length_hist"]=np.histogram(lens.clip(upper=400),bins=40)[0].tolist()
print("\ndataset stats captured.")

```

label	n	pos%	rho
toxic	15294	9.58	9.4
severe_toxic	1595	1.00	99.0
obscene	8449	5.29	17.9
threat	478	0.30	332.8
insult	7877	4.94	19.3
identity_hate	1405	0.88	112.6

cardinality=0.2200 density=0.0367 zero-label frac=0.8983
 avg pos rate=3.67% naive all-neg accuracy=96.33%
 token length: median=36 p95=229 p99=567
 chosen tmax=150 covers 89.9% without truncation

dataset stats captured.

6 · Models, losses, metrics

```

from tensorflow.keras import layers, models, regularizers, backend as K
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.metrics import (precision_recall_fscore_support,
                             average_precision_score, roc_auc_score)

eps=K.epsilon()

def bce(y,p):
    p=K.clip(p,eps,1-eps); return -K.mean(y*K.log(p)+(1-y)*K.log(1-p))
def alpha_bce(alpha):
    def loss(y,p):
        p=K.clip(p,eps,1-eps); a=alpha*y+(1-alpha)*(1-y)
        return -K.mean(a*(y*K.log(p)+(1-y)*K.log(1-p)))
    return loss
def focal(gamma,alpha):
    def loss(y,p):
        p=K.clip(p,eps,1-eps); pt=y*p+(1-y)*(1-p); at=alpha*y+(1-alpha)*(1-y)
        return -K.mean(at*K.pow(1-pt,gamma)*K.log(pt))
    return loss

def build(name,vocab,d,T,loss_fn,seed):
    tf.random.set_seed(seed)
    inp=layers.Input(shape=(T,)); emb=layers.Embedding(vocab,d,mask_zero=True)(inp)
    if name=="vanilla_ff":
        x=layers.GlobalAveragePooling1D()(emb); x=layers.Dense(10,activation="relu")(x)
    elif name=="lstm_tiny_10_5":
        x=layers.LSTM(10,return_sequences=True)(emb); x=layers.LSTM(5)(x)
    elif name=="rnn_small_20_10":
        x=layers.LSTM(20,return_sequences=True)(emb); x=layers.LSTM(10)(x)

```

```

elif name=="lstm_60_50":
    x=layers.LSTM(60,return_sequences=True)(emb); x=layers.LSTM(50)(x)
elif name=="proposed_lstm_50_dense_25":
    x=layers.LSTM(50)(emb); x=layers.Dropout(0.3)(x)
    x=layers.Dense(25,activation="relu",kernel_regularizer=regularizers.l2(1e-5))(x)
else: raise ValueError(name)
out=layers.Dense(L,activation="sigmoid")(x)
m=models.Model(inp,out)
m.compile(optimizer=tf.keras.optimizers.Adam(1e-3, clipnorm=1.0), loss=loss_fn)
return m

def tune_thr(yv,pv):
    thr=np.full(L,0.5)
    for l in range(L):
        bf,bt=-1,0.5
        for t in np.linspace(0.05,0.95,19):
            _,f,_=precision_recall_fscore_support(yv[:,l],(pv[:,l]>=t).astype(int),
                                                  average="binary",zero_division=0)
            if f>bf: bf,bt=f,t
        thr[l]=bt
    return thr

def evaluate(yt,pp,thr):
    pred=(pp>=thr).astype(int)
    P,Rc,F,_=precision_recall_fscore_support(yt,pred,average=None,zero_division=0)
    per={}
    for l,lab in enumerate(LABELS):
        try: ap=float(average_precision_score(yt[:,l],pp[:,l]))
        except: ap=float("nan")
        try: rc=float(roc_auc_score(yt[:,l],pp[:,l]))
        except: rc=float("nan")
        per[lab]=dict(P=float(P[l]),R=float(Rc[l]),F1=float(F[l]),AP=ap,ROC=rc)
    _,Fmi,_=precision_recall_fscore_support(yt,pred,average="micro",zero_division=0)
    _,Fw,_=precision_recall_fscore_support(yt,pred,average="weighted",zero_division=0)
    return dict(per_class=per, macro_f1=float(np.mean(F)),
                macro_prauc=float(np.nanmean([per[x]["AP"] for x in LABELS])),
                macro_roc=float(np.nanmean([per[x]["ROC"] for x in LABELS])),
                micro_f1=float(Fmi), weighted_f1=float(Fw),
                accuracy=float((pred==yt).mean()))

print("helpers ready.")

```

helpers ready.

7 · Run all experiments (*auto-saves + resumes — safe to re-run after a disconnect*)

```

import json, time
from iterstrat.ml_stratifiers import MultilabelStratifiedShuffleSplit as MSSS

STORE="results.json"
def load():
    if os.path.exists(STORE):
        return json.load(open(STORE))
    return {"env":ENV,"config":CONFIG,"stats":STATS,"runs":{}}
def save(R): json.dump(R,open(STORE,"w"),indent=2)
R=load(); R["env"]=ENV; R["config"]=CONFIG; R["stats"]=STATS

proposed="proposed_lstm_50_dense_25"
MAIN=["vanilla_ff","lstm_tiny_10_5","rnn_small_20_10","lstm_60_50",proposed]
JOBS=[(a,"focal" if a==proposed else "bce") for a in MAIN] + \
      [(proposed,"bce"),(proposed,"alpha_bce")]

def split(Ysub,frac,seed):
    tr,te=next(MSSS(n_splits=1,test_size=frac,random_state=seed).split(np.zeros(len(Ysub)),Ysub))
    return tr,te

total_jobs=CONFIG["seeds"]*len(JOBS); done=0
print(f"Training {total_jobs} model-runs total ({len(JOBS)} configs x {CONFIG['seeds']} seeds)\n")

for seed in range(CONFIG["seeds"]):
    tr,te=split(Y,0.20,seed)
    trX,teX=df["clean"].values[tr],df["clean"].values[te]
    teRaw=df["comment_text"].values[te]

```

```

Ytr_all,Yte=Y[tr],Y[te]
tri,vai=split(Ytr_all,0.10,seed)
tok=Tokenizer(num_words=CONFIG["vocab"],oov_token="<UNK>"); tok.fit_on_texts(trX[tri])
enc=lambda t: pad_sequences(tok.texts_to_sequences(t),maxlen=CONFIG["tmax"],
                             padding="post",truncating="post")
Xtr,Xval,Xte=enc(trX[tri]),enc(trX[vai]),enc(teX)
Ytr,Yval=Ytr_all[tri],Ytr_all[vai]

if "naive" not in R["runs"]:
    pred=np.zeros_like(Yte)
    R["runs"]["naive"]={"accuracy":float((pred==Yte).mean()),"macro_f1":0.0,
                       "micro_f1":0.0,"macro_roc":0.5,
                       "macro_prauc":float(np.nanmean(
                           [average_precision_score(Yte[:,l],np.zeros(len(Yte)))
                            if Yte[:,l].sum() else float("nan") for l in range(L)]))}
    save(R)

for arch,lk in JOBS:
    key=f"{arch}|{lk}|seed{seed}"
    if key in R["runs"]:
        done+=1; print(f"[{done}/{total_jobs}] skip (already done) {key}"); continue
    lf = bce if lk=="bce" else (alpha_bce(CONFIG["alpha"]) if lk=="alpha_bce"
                               else focal(CONFIG["gamma"],CONFIG["alpha"]))
    m=build(arch,CONFIG["vocab"],CONFIG["embdim"],CONFIG["tmax"],lf,seed)
    t0=time.time()
    m.fit(Xtr,Ytr,validation_data=(Xval,Yval),epochs=CONFIG["epochs"],
          batch_size=CONFIG["batch"],verbose=0,
          callbacks=[EarlyStopping(monitor="val_loss",patience=3,restore_best_weights=True)])
    train_min=(time.time()-t0)/60
    pv=m.predict(Xval,batch_size=512,verbose=0); thr=tune_thr(Yval,pv)
    t1=time.time(); pt=m.predict(Xte,batch_size=512,verbose=0); infer_s=time.time()-t1
    res=evaluate(Yte,pt,thr)
    res.update(train_min=float(train_min),infer_s=float(infer_s),
               n_params=int(m.count_params()))
    if arch==proposed and lk=="focal" and seed==0:
        pos=np.where(Yte.sum(1)>0)[0][:8]; neg=np.where(Yte.sum(1)==0)[0][:4]
        R["examples"]=[dict(text=str(teRaw[ix]))[:300],
                        gold={LABELS[l]:int(Yte[ix,l]) for l in range(L)},
                        prob={LABELS[l]:round(float(pt[ix,l]),3) for l in range(L)},
                        thr={LABELS[l]:round(float(thr[l]),2) for l in range(L)}
                        for ix in list(pos)+list(neg)]
    R["runs"][key]=res; save(R); done+=1
    print(f"[{done}/{total_jobs}] seed{seed} {arch:26s} {lk:9s} "
          f"macroF1={res['macro_f1']:.4f} PRAUC={res['macro_prauc']:.4f} "
          f"acc={100*res['accuracy']:.3f} ({train_min:.1f} min)")
    K.clear_session()

print("\nALL DONE - results.json written.")

```

Training 21 model-runs total (7 configs x 3 seeds)

[1/21]	seed0 vanilla_ff	bce	macroF1=0.5102	PRAUC=0.5106	acc=97.572	(0.4 min)
[2/21]	seed0 lstm_tiny_10_5	bce	macroF1=0.5079	PRAUC=0.4980	acc=97.590	(1.4 min)
[3/21]	seed0 rnn_small_20_10	bce	macroF1=0.5192	PRAUC=0.5185	acc=97.514	(1.1 min)
[4/21]	seed0 lstm_60_50	bce	macroF1=0.5182	PRAUC=0.5235	acc=97.809	(1.4 min)
[5/21]	seed0 proposed_lstm_50_dense_25	focal	macroF1=0.5172	PRAUC=0.5215	acc=97.608	(0.7 min)
[6/21]	seed0 proposed_lstm_50_dense_25	bce	macroF1=0.5129	PRAUC=0.5092	acc=97.408	(0.9 min)
[7/21]	seed0 proposed_lstm_50_dense_25	alpha_bce	macroF1=0.5049	PRAUC=0.5183	acc=97.777	(0.7 min)
[8/21]	seed1 vanilla_ff	bce	macroF1=0.4950	PRAUC=0.5050	acc=97.013	(0.4 min)
[9/21]	seed1 lstm_tiny_10_5	bce	macroF1=0.5026	PRAUC=0.4988	acc=97.285	(1.3 min)
[10/21]	seed1 rnn_small_20_10	bce	macroF1=0.5075	PRAUC=0.5099	acc=97.259	(1.3 min)
[11/21]	seed1 lstm_60_50	bce	macroF1=0.5195	PRAUC=0.5281	acc=97.439	(1.3 min)
[12/21]	seed1 proposed_lstm_50_dense_25	focal	macroF1=0.5051	PRAUC=0.5183	acc=96.823	(0.9 min)
[13/21]	seed1 proposed_lstm_50_dense_25	bce	macroF1=0.5043	PRAUC=0.5074	acc=97.658	(0.9 min)
[14/21]	seed1 proposed_lstm_50_dense_25	alpha_bce	macroF1=0.5031	PRAUC=0.5219	acc=97.670	(1.2 min)
[15/21]	seed2 vanilla_ff	bce	macroF1=0.5053	PRAUC=0.5053	acc=97.571	(0.4 min)
[16/21]	seed2 lstm_tiny_10_5	bce	macroF1=0.5196	PRAUC=0.5179	acc=97.687	(1.3 min)
[17/21]	seed2 rnn_small_20_10	bce	macroF1=0.5224	PRAUC=0.5103	acc=97.435	(1.3 min)
[18/21]	seed2 lstm_60_50	bce	macroF1=0.5078	PRAUC=0.5078	acc=97.047	(1.0 min)
[19/21]	seed2 proposed_lstm_50_dense_25	focal	macroF1=0.4988	PRAUC=0.5092	acc=96.759	(0.9 min)
[20/21]	seed2 proposed_lstm_50_dense_25	bce	macroF1=0.4933	PRAUC=0.5104	acc=97.440	(0.7 min)
[21/21]	seed2 proposed_lstm_50_dense_25	alpha_bce	macroF1=0.5082	PRAUC=0.5176	acc=97.723	(0.8 min)

ALL DONE - results.json written.

✓ 8 · FINAL REPORT (*copy this whole block back, or just download results.json*)

```

from tensorflow.keras import layers, models, regularizers, backend as K
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.metrics import (precision_recall_fscore_support,
                             average_precision_score, roc_auc_score)

eps=K.epsilon()

def bce(y,p):
    p=K.clip(p,eps,1-eps); return -K.mean(y*K.log(p)+(1-y)*K.log(1-p))
def alpha_bce(alpha):
    def loss(y,p):
        p=K.clip(p,eps,1-eps); a=alpha*y+(1-alpha)*(1-y)
        return -K.mean(a*(y*K.log(p)+(1-y)*K.log(1-p)))
    return loss
def focal(gamma,alpha):
    def loss(y,p):
        p=K.clip(p,eps,1-eps); pt=y*p+(1-y)*(1-p); at=alpha*y+(1-alpha)*(1-y)
        return -K.mean(at*K.pow(1-pt,gamma)*K.log(pt))
    return loss

def build(name,vocab,d,T,loss_fn,seed):
    tf.random.set_seed(seed)
    inp=layers.Input(shape=(T,)); emb=layers.Embedding(vocab,d)(inp) # no mask_zero -> cuDNN-safe
    if name=="vanilla_ff":
        x=layers.GlobalAveragePooling1D()(emb); x=layers.Dense(10,activation="relu")(x)
    elif name=="lstm_tiny_10_5":
        x=layers.LSTM(10,return_sequences=True)(emb); x=layers.LSTM(5)(x)
    elif name=="rnn_small_20_10":
        x=layers.LSTM(20,return_sequences=True)(emb); x=layers.LSTM(10)(x)
    elif name=="lstm_60_50":
        x=layers.LSTM(60,return_sequences=True)(emb); x=layers.LSTM(50)(x)
    elif name=="proposed_lstm_50_dense_25":
        x=layers.LSTM(50)(emb); x=layers.Dropout(0.3)(x)
        x=layers.Dense(25,activation="relu",kernel_regularizer=regularizers.l2(1e-5))(x)
    else: raise ValueError(name)
    out=layers.Dense(L,activation="sigmoid")(x)
    m=models.Model(inp,out); m.compile(optimizer=tf.keras.optimizers.Adam(1e-3),loss=loss_fn)
    return m

def tune_thr(yv,pv):
    thr=np.full(L,0.5)
    for l in range(L):
        bf,bt=-1,0.5
        for t in np.linspace(0.05,0.95,19):
            _,_,f,_=precision_recall_fscore_support(yv[:,l],(pv[:,l]>=t).astype(int),
                                                    average="binary",zero_division=0)
            if f>bf: bf,bt=f,t
        thr[l]=bt
    return thr

def evaluate(yt,pp,thr):
    pred=(pp>=thr).astype(int)
    P,Rc,F,_=precision_recall_fscore_support(yt,pred,average=None,zero_division=0)
    per={}
    for l,lab in enumerate(LABELS):
        try: ap=float(average_precision_score(yt[:,l],pp[:,l]))
        except: ap=float("nan")
        try: rc=float(roc_auc_score(yt[:,l],pp[:,l]))
        except: rc=float("nan")
        per[lab]=dict(P=float(P[l]),R=float(Rc[l]),F1=float(F[l]),AP=ap,ROC=rc)
    _,_,Fmi,_=precision_recall_fscore_support(yt,pred,average="micro",zero_division=0)
    _,_,Fw,_=precision_recall_fscore_support(yt,pred,average="weighted",zero_division=0)
    return dict(per_class=per, macro_f1=float(np.mean(F)),
               macro_prauc=float(np.nanmean([per[x]["AP"] for x in LABELS])),
               macro_roc=float(np.nanmean([per[x]["ROC"] for x in LABELS])),
               micro_f1=float(Fmi), weighted_f1=float(Fw),
               accuracy=float((pred==yt).mean()))
print("helpers ready (cuDNN-safe).")

```

helpers ready (cuDNN-safe).

9 · Download results.json

```
from google.colab import files  
files.download("results.json")
```

```
import os  
print("exists:", os.path.exists("results.json"))  
!ls -la results.json 2>/dev/null
```

```
exists: False
```

```
!rm -f results.json
```